EPFL

Embedded Systems Design

Charlotte Heibig
Ismael Frei

EPFL - 07.06.2024

Overview

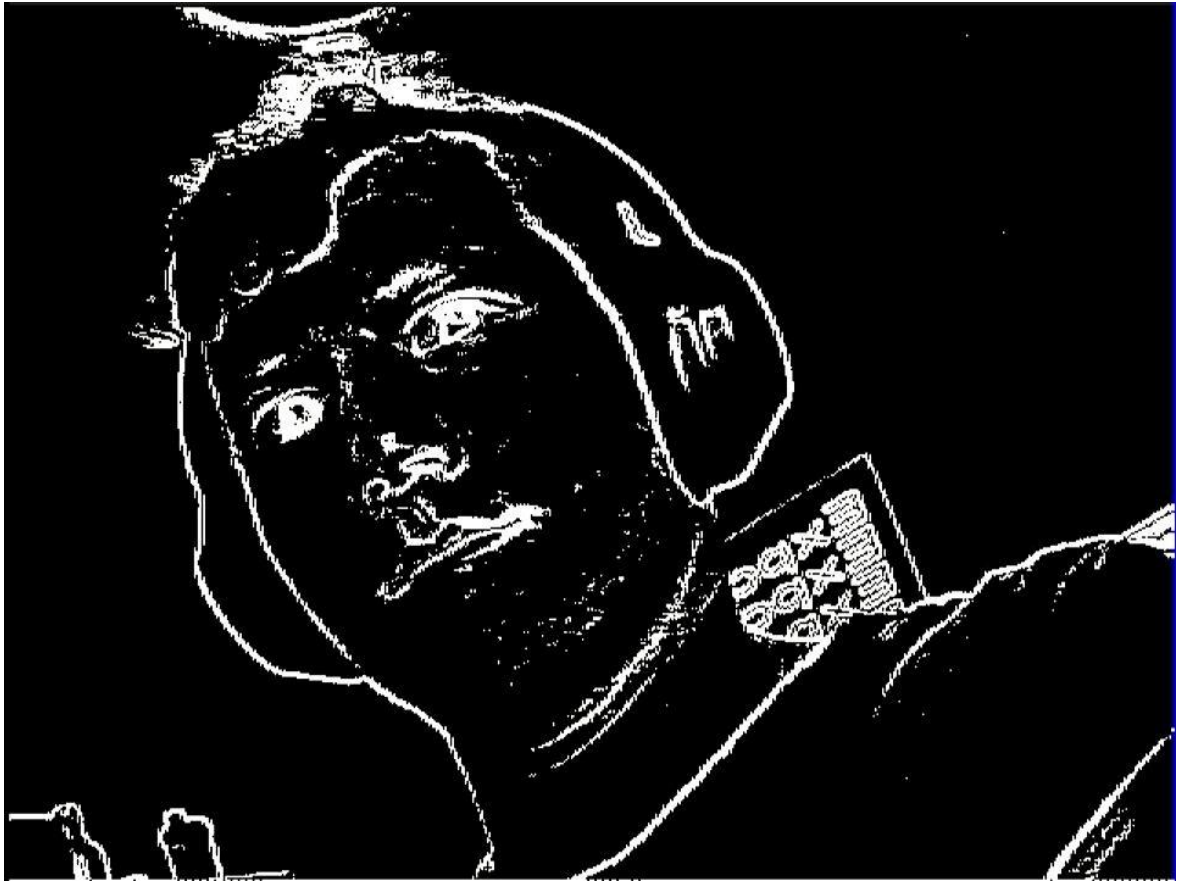
- Part I : Software Implementation
- Part II : Hardware accelerator
- Part III : Profiling
- Part IV : Further optimization
- Part V : Dynamic Thresholding Feature
- Demo
- Questions

Part I: Sobel edge detection algorithm

Charlotte Heibig & Ismael Frei

- Apply kernels to each pixel and its 8 surrounding pixels to detect intensity differences in every direction
- Identify where the intensity changes rapidly according to a certain threshold to detect edges
- Results in a new black and white image with only edges
- Used already provided code

Result



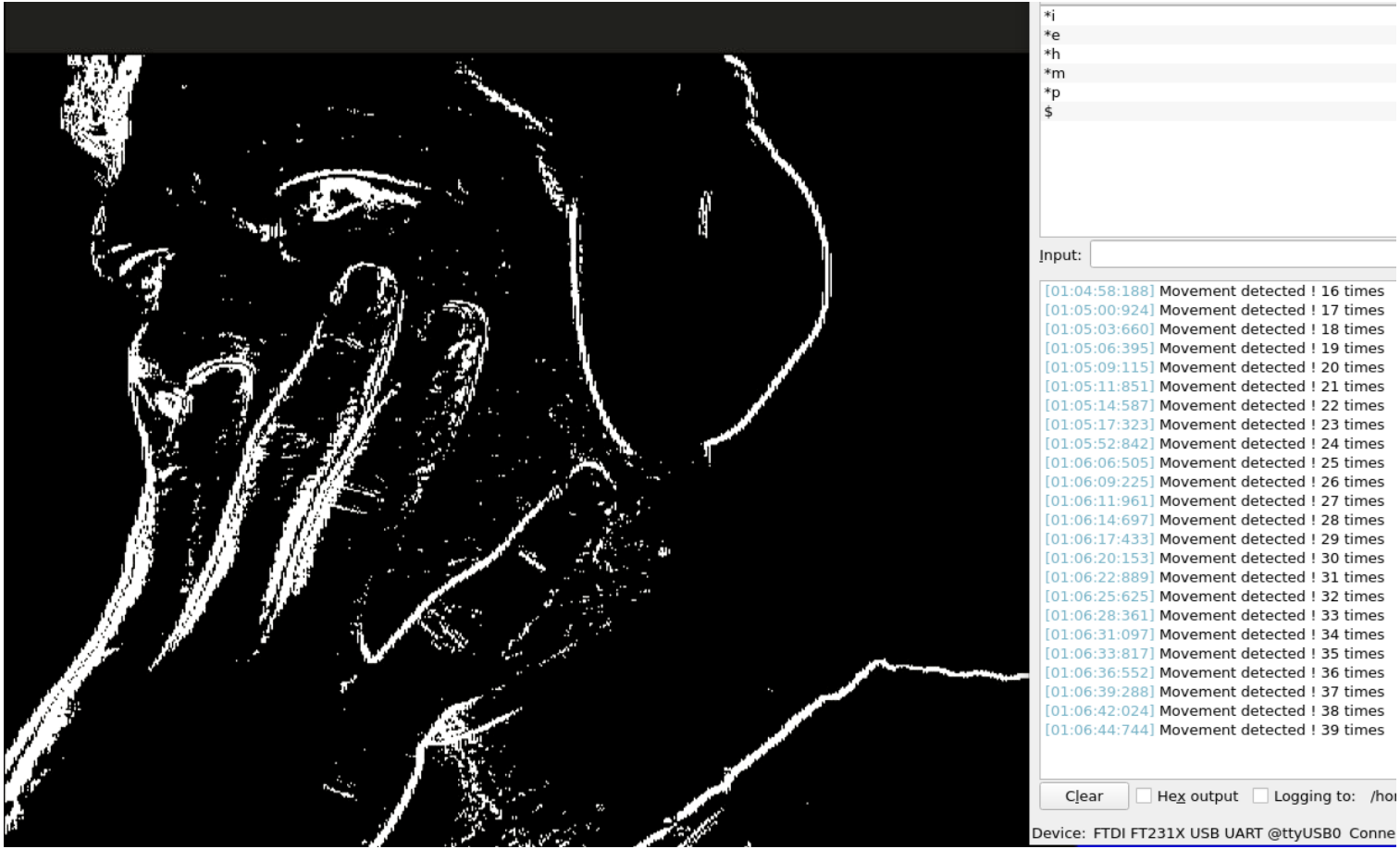
Charlotte Heibig & Ismael Frei

Movement detection using Sobel

Charlotte Heibig & Ismael Frei

- Take a grayscale picture
- Perform the Sobel algorithm and store the result in memory
- Take a second picture and perform the Sobel algorithm
- Compare the two results pixel by pixel
- If the result is different from 0, then the edge has changed, given a certain threshold (so that movement is only detected after a certain number of pixel changes)

Result



Charlotte Heibig & Ismael Frei

Part II: Sobel as a Custom Instruction

```
module sobelCi #( parameter[7:0] customId = 8'h00 )
                ( input wire        start,
                  input wire [31:0] valueA,
                                    valueB,
                  input wire [7:0]  ciN,
                  output wire       done,
                  output wire [31:0] result );

    wire s_isMyCi = (ciN == customId) ? start : 1'b0;
    assign done = s_isMyCi;
    /* SobelCi takes 8 pixels (8 neighbours of a pixel) as input and produ
     * The pixels come in valueA and valueB row wise from top left pixel t
     * Middle pixel is not needed for sobel algorithm as it is always mult
     *
     */
    wire[7:0] pixel0in = valueA[31:24];
    wire[7:0] pixel1in = valueA[23:16];
    wire[7:0] pixel2in = valueA[15:8];
    wire[7:0] pixel3in = valueA[7:0];
    wire[7:0] pixel4in = valueB[31:24];
    wire[7:0] pixel5in = valueB[23:16];
    wire[7:0] pixel6in = valueB[15:8];
    wire[7:0] pixel7in = valueB[7:0];

    wire[16:0] dX, dXplus, dXminus, dY, dYplus, dYminus;

    // Sobel operator
    assign dXplus = pixel2in + pixel4in + pixel4in + pixel7in;
    assign dXminus = pixel0in +  pixel3in + pixel3in + pixel5in;
    //take absolute value of difference
    assign dX = (dXplus > dXminus) ? dXplus - dXminus : dXminus - dXplus;

    assign dYplus = pixel0in + pixel1in + pixel1in + pixel2in;
    assign dYminus = pixel5in + pixel6in + pixel6in + pixel7in;
    //take absolute value of difference
    assign dY = (dYplus > dYminus) ? dYplus - dYminus : dYminus - dYplus;

    assign result = s_isMyCi ? {16'b0, dX + dY} : 32'b0;
endmodule
```

- A simple custom Instruction can be created for sobel
- valueA and valueB store 8 bytes -> perfect for 8 neighbouring pixels
- No registers or clock, purely combinational logic

Charlotte Heibig & Ismael Frei

Result

Charlotte Heibig & Ismael Frei

Comparison of the Sobel algorithm : hardware and software implementation

- Reduces the CPU cycles by a factor of 3.25
- Reduces the CPU stall cycles by a factor of 3.15
- Reduces CPU bus-idle cycles by a factor of 3.38
- Still room for improvement !

Results	cycles without HW acceleration	cycles with HW acceleration
CPU execution cycles	334'856'828	102'773'340
CPU stall cycles	255'656'675	81'355'177
CPU bus-idle cycles	155'509'118	46'019'168

Charlotte Heibig & Ismael Frei

Movement detection algorithm HW implementation

- Reduces CPU cycles by a factor of 1.1
- 10x faster than Sobel -> not really relevant

Results	cycles without HW acceleration	cycles with HW acceleration
CPU execution cycles	24'880'570	22'525'469
CPU stall cycles	18'655'980	18'378'947
CPU bus-idle cycles	11'741'333	9'698'014

Charlotte Heibig & Ismael Frei

Part IV: Sobel algorithm optimization

Charlotte Heibig & Ismael Frei

- Use DMA controller and SRAM as a “ping peng pang pong buffer” (divide SRAM into 4 buffers)
- 3 rows are in the buffer and worked with while one new row is added to the buffer in the background
- Each buffer needs 640 bytes of space, which means that in total we need 640 words for the four buffers (2560 bytes)

Dual port SSRAM

- Problem: Pixels are saved as bytes inside words. Only entire words can be accessed.
- Solution: customized SSRAM that lets user access Words with byte offset.
- Example: Word 1 with offset 3 outputs {W1B3,W3B0:B2}

Word 0	Word 1	Word 2	Word 3
B0 B1 B2 B3	B0 B1 B2 B3	B0 B1 B2 B3	B0 B1 B2 B3

Charlotte Heibig & Ismael Frei

Comparison of the Sobel algorithm with and without DMA

- Reduces CPU cycles by a factor of 15 !
- Reduces the CPU stall cycles by a factor of 13.9
- Reduces CPU bus-idle cycles by a factor of 16.2
- Now the movement detection algorithm and Sobel operate at the same speed !

Results	cycles without HW acceleration	cycles with HW acceleration	cycles with optimized HW acceleration
CPU execution cycles	334'856'828	101'867'190	22'353'305
CPU stall cycles	255'656'675	80'459'872	18'282'876
CPU bus-idle cycles	155'509'118	46'030'981	9'571'337

Charlotte Heibig & Ismael Frei

Part V: Dynamic Thresholding Feature

Too much edges detected:

- increase threshold

Not much edges detected

- decrease threshold

Charlotte Heibig & Ismael Frei

Questions ?